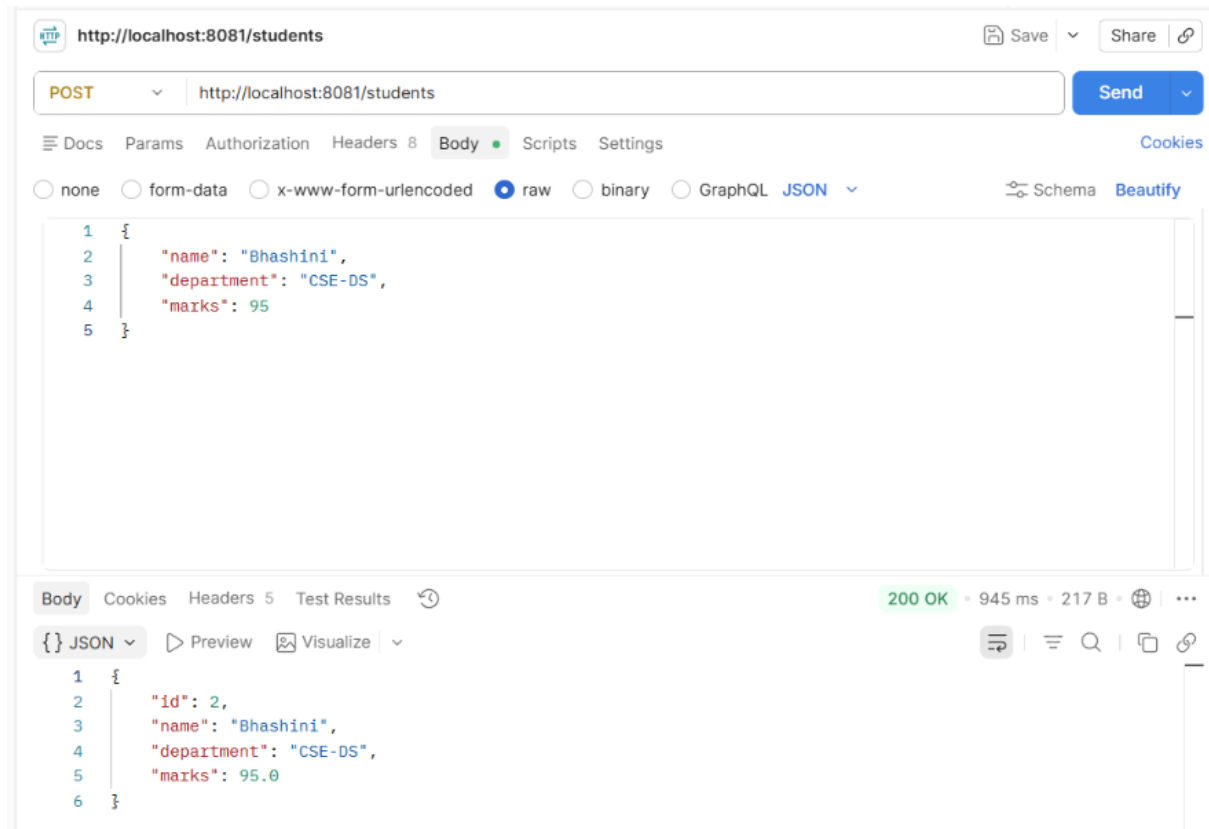


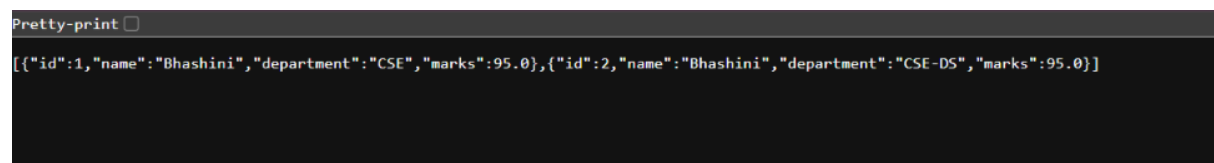
Spring Data JPA with Spring Boot, Hibernate

1. spring-data-jpa-handson - Spring Data JPA - Quick Example

Operation : **CREATE** , Method : **Post**



Result:



Operation : **READ BY ID** , Method : **Get**

http://localhost:8081/students/1

GET http://localhost:8081/students/1

Body

```
1 {
2   "name": "Bhashini",
3   "department": "Data Science",
4   "marks": 98
5 }
```

200 OK • 30 ms • 223 B •

Body

```
1 {
2   "id": 1,
3   "name": "Bhashini",
4   "department": "Data Science",
5   "marks": 98.0
6 }
```

RESULT :

```
Pretty-print {
  "id": 1,
  "name": "Bhashini",
  "department": "Data Science",
  "marks": 98.0
}
```

Operation : **UPDATE** , Method : **PUT**

http://localhost:8081/students/1

PUT http://localhost:8081/students/1

Body

```
1 {
2   "name": "Bhashini",
3   "department": "Data Science",
4   "marks": 98
5 }
```

200 OK • 66 ms • 223 B •

Body

```
1 {
2   "id": 1,
3   "name": "Bhashini",
4   "department": "Data Science",
5   "marks": 98.0
6 }
```

RESULT :

```
Pretty-print ☐

{"id":1,"name":"Bhashini","department":"Data Science","marks":98.0}
```

Operation : **DELETE** , Method : **DELETE**

The screenshot shows a REST client interface with the URL `http://localhost:8081/students/1`. The method is set to **DELETE**. The request body is a JSON object: `{ "name": "Bhashini", "department": "Data Science", "marks": 98 }`. The response status is **200 OK** with a response time of 973 ms and a body size of 193 B. The response body is `1 Student deleted successfully!`.

The screenshot shows the same REST client interface with the URL `http://localhost:8081/students/1`. The method is set to **GET**. The response status is **200 OK** with a response time of 50 ms and a body size of 123 B. The response body is a JSON object: `{ "name": "Bhashini", "department": "Data Science", "marks": 98 }`.

1. spring-data-jpa-handson - Difference between JPA, Hibernate and Spring Data JPA

Difference between JPA, Hibernate, and Spring Data JPA

JPA (Java Persistence API) is a Java specification that defines a standard way to store, retrieve, update, and delete data from a database using Java objects. It only provides rules and interfaces, but it does not contain the actual implementation.

Hibernate is one of the most popular implementations of JPA. It is an ORM (Object Relational Mapping) framework that converts Java objects into database tables and vice versa. Hibernate handles SQL generation, database connections, and CRUD operations automatically, reducing the amount of manual SQL code developers need to write.

Spring Data JPA is built on top of JPA and Hibernate. It provides an additional abstraction layer that removes most of the boilerplate code. Instead of writing DAO classes and many database queries, developers can simply create repository interfaces like `JpaRepository`, and Spring Data JPA automatically provides CRUD operations and query methods.

Difference between Hibernate and Spring Data JPA

Hibernate	Spring Data JPA
ORM framework and JPA implementation	Abstraction built on top of JPA/Hibernate
Requires more configuration and coding	Requires very little code
Developers often write DAO classes manually	Repository interfaces handle CRUD automatically
Gives more control over Hibernate features	Best for rapid development with less boilerplate

In simple words

- **JPA** = Rules or specification.
- **Hibernate** = Tool that follows those rules and performs database operations.
- **Spring Data JPA** = Makes Hibernate even easier by automatically generating most of the database code.

Example from our project

In our Student CRUD application:

- We used **JPA annotations** like `@Entity`, `@Table`, and `@Id`.
- **Hibernate** converted the Student Java class into the students table and executed SQL queries behind the scenes.
- **Spring Data JPA** allowed us to perform CRUD operations by simply extending `JpaRepository`, without writing SQL queries manually.

This combination makes Java applications easier to develop, maintain, and scale.